

FleetPulse: Scalable IoT Fleet Management for Driverless Taxis

Final Project Report | Software Architecture and Scalability

Student: [Your full name]

Student ID: [Your student ID]

Submission date: 1 October 2026

Executive Summary

FleetPulse is a proposed smart-city platform for monitoring a fleet of driverless taxis through IoT telemetry. Each vehicle periodically reports its identity, time, location, speed, battery, temperature and operating status. A lightweight MQTT connection carries the events to a broker; independent consumers validate, store and act on them. The design separates fleet monitoring, alerts, bookings and analytics so each workload can scale without scaling the whole application.

The project has a working local proof of concept: a Node.js publisher sends a JSON message for vehicle VT001 to a Mosquitto MQTT broker. The repository also contains an importable Node-RED flow intended to parse and display the message. The publisher-to-broker path was confirmed; the saved status update records a Node-RED flow problem, and no cloud deployment or measured fleet load test is evidenced. This report therefore separates demonstrated results from the production design and its proposed acceptance tests.

1. Problem and Objectives

A city fleet needs timely visibility of vehicle health and availability without tightly coupling every taxi to every backend feature. Periodic polling creates avoidable network traffic, while a single application that owns telemetry, bookings and analytics becomes harder to scale and recover independently. FleetPulse addresses this with event-driven telemetry and independently deployable processing services.

The project objectives are to:

- Define a secure, versioned telemetry contract and MQTT topic structure.
- Demonstrate a simulated taxi publishing structured telemetry to a local broker.
- Design independent services for monitoring, alerts, booking and analytics.
- Explain how ingestion, storage and consumers can scale horizontally.
- Define reproducible tests and measurable criteria for a larger fleet.

The prototype is a software simulation, not an autonomous-driving system. It does not control a real vehicle, make safety-critical driving decisions, or demonstrate regulatory certification.

2. Architecture and Data Flow

The target architecture is event-driven. Vehicle simulators (and, in a future real deployment, authenticated vehicle gateways) publish telemetry to an MQTT broker. The broker routes the event to a stateless ingestion service. That service validates the schema, rejects invalid or stale messages, and forwards accepted events to a durable queue or stream. Separate consumers update the latest vehicle state, persist history, evaluate alerts and feed analytics. A booking API reads availability and emits booking events; it does not send driving commands.

Telemetry path: Vehicle → MQTT broker → ingestion and validation → durable event stream → monitoring / alerts / history / analytics.

Booking path: Passenger client → booking API → booking store and dispatch event → fleet availability service.

For a managed cloud deployment, AWS IoT Core is a candidate MQTT endpoint, with queue/stream services decoupling ingestion from consumers, a time-series or relational store selected by query needs, and managed metrics/logging for operations. These are design choices, not deployed resources in this submission. The local proof of concept uses Mosquitto and Node-RED to keep the first integration test inexpensive and reproducible.

Telemetry contract

The prototype message contains vehicleId, an ISO-8601 timestamp, latitude, longitude, speed, battery level,

temperature and status. A production contract should add a schema version, unique event ID, units, and validation bounds. Consumers should be idempotent because QoS 1 delivery can repeat a message. The topic convention `taxis/vehicle/{vehicleId}/telemetry` allows a service to subscribe to one vehicle or use `taxis/vehicle/+/telemetry` for the fleet. Separate command and telemetry topics prevent a monitoring consumer from accidentally becoming a vehicle-control path.

3. Scalability, Reliability and Security

The services are stateless where possible and can be scaled horizontally using queue lag, processing latency and CPU as signals. The broker and durable stream form explicit capacity boundaries; consumers can catch up independently after a restart. Time-series retention and downsampling keep historical storage bounded, while a latest-state store supports fast availability queries. Partitioning by vehicle ID preserves per-vehicle ordering where required. Multi-zone broker, queue and data-store configurations are required before production.

An illustrative load target is 1,000 simulated vehicles publishing one 512-byte payload every five seconds: 200 messages/second and approximately 102 KB/second of payload before protocol overhead. At 10,000 vehicles the same rate is 2,000 messages/second and approximately 1.0 MB/second. These calculations size a test, not measured project performance. The local prototype has not been load-tested; the broker, network and storage limits must be measured before selecting production capacity.

Reliability measures include MQTT QoS 1 for events that must arrive, bounded retries with backoff, a dead-letter path for invalid events, message timestamps, broker last-will/offline status, and alerting for stale telemetry. Telemetry can tolerate occasional delay; booking state and safety-related signals need explicit freshness and availability requirements. The platform must fail closed for any future command workflow and must never treat missing telemetry as proof that a vehicle is safe.

Security design requires unique device identity, TLS, per-vehicle topic ACLs, managed secrets/certificates, least-privilege service roles and audit logs. Location and trip data are sensitive: minimize collected fields, restrict access, define retention/deletion periods and avoid exposing exact location in general analytics. Production rollout also requires threat modelling, privacy review, incident procedures and applicable transport regulation review.

4. Implementation and Evaluation

The implemented slice is intentionally small: `index.js` creates one telemetry object, connects to `mqtt://localhost:1883`, publishes to `taxis/vehicle/VT001/telemetry`, reports success or connection failure, and closes the client. `package.json` declares `MQTT.js` and an npm start command. `node-red-flow.json` describes MQTT input, JSON parsing and debug output against the same local broker.

The recorded terminal evidence confirms Connected to MQTT broker and Published message to: `taxis/vehicle/VT001/telemetry`. This verifies message publication from the simulator to Mosquitto. The project status update also records a Node-RED Debug message, No url specified; the corrected flow file exists, but the end-to-end Node-RED display was not confirmed. No evidence is available for continuous publishing, multi-vehicle simulation, persistence, cloud hosting or benchmark results. The distinction is important: the architecture is scalable by design, but scalability has not yet been empirically demonstrated.

The next evaluation should use a repeatable simulator with controlled rates and vehicle counts of 1, 10, 100 and 1,000. Record publish-to-consumer latency (p50/p95/p99), accepted and rejected message counts, message loss/duplicates, consumer lag, CPU/memory, reconnect time and storage growth. Proposed acceptance criteria for the 1,000-vehicle test are at least 99.9% accepted valid events, p95 end-to-end latency below two seconds under steady load, no unbounded queue growth, and successful recovery from a consumer restart without losing durable events. These are future test gates, not achieved results. The Node-RED issue should be resolved and captured before claiming the complete local path.

5. Conclusion

FleetPulse provides a concrete, extensible idea for a scalable IoT taxi fleet: publish small vehicle events over MQTT, validate and buffer them, then scale monitoring, alerts, booking and analytics independently. The repository demonstrates the first integration step with a real local MQTT publish and includes the flow and configuration needed to continue the prototype. The major evidence gap is downstream processing and measured scale. A successful Node-RED run followed by controlled load, resilience and security tests is required before claiming a deployed or performance-validated system. The design deliberately excludes real vehicle control; that would require a separate safety-critical

engineering and regulatory programme.

Appendix A. Implemented Publisher

The following is the current index.js source included in the project:

```
const mqtt = require("mqtt");

const brokerUrl = "mqtt://localhost:1883";
const topic = "taxis/vehicle/VT001/telemetry";

const telemetry = {
  vehicleId: "VT001",
  timestamp: new Date().toISOString(),
  latitude: 51.5074,
  longitude: -0.1278,
  speed: 32.5,
  batteryLevel: 87,
  temperature: 24.6,
  status: "available"
};

const client = mqtt.connect(brokerUrl);

client.on("connect", () => {
  console.log("Connected to MQTT broker");

  client.publish(topic, JSON.stringify(telemetry), { qos: 0 }, (error) => {
    if (error) {
      console.error("Publish failed:", error.message);
      client.end();
      process.exitCode = 1;
      return;
    }

    console.log("Published message to: ${topic}");
    console.log(JSON.stringify(telemetry, null, 2));
    client.end();
  });
});

client.on("error", (error) => {
  console.error("MQTT connection error:", error.message);
  client.end();
  process.exitCode = 1;
});
```

Appendix B. Configuration and Evidence

package.json:

```
{
  "name": "driverless-taxi-system",
  "version": "1.0.0",
  "private": true,
  "description": "Minimal simulated driverless taxi MQTT publisher",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "report:docx": "node generate-project-report-docx.js",
    "report:pdf": "swift generate-project-report-pdf.swift"
  },
  "dependencies": {
    "docx": "^9.7.2",
    "mqtt": "^5.14.1"
  }
}
```

node-red-flow.json: import this flow into Node-RED and deploy it to reproduce the intended local MQTT-to-Debug path.

```
[
  {
    "id": "mqtt-input-vehicle",
    "type": "mqtt in",
    "z": "driverless-taxi-flow",
```

```

    "name": "Taxi telemetry",
    "topic": "taxis/vehicle/VT001/telemetry",
    "qos": "0",
    "datatype": "utf8",
    "broker": "local-mosquitto",
    "nl": false,
    "rap": true,
    "rh": 0,
    "inputs": 0,
    "x": 180,
    "y": 120,
    "wires": [["json-parser"]]
  },
  {
    "id": "json-parser",
    "type": "json",
    "z": "driverless-taxi-flow",
    "name": "Parse JSON",
    "property": "payload",
    "action": "",
    "pretty": false,
    "x": 370,
    "y": 120,
    "wires": [["debug-output"]]
  },
  {
    "id": "debug-output",
    "type": "debug",
    "z": "driverless-taxi-flow",
    "name": "Vehicle telemetry",
    "active": true,
    "tosidebar": true,
    "console": false,
    "tostatus": false,
    "complete": "payload",
    "targetType": "msg",
    "x": 590,
    "y": 120,
    "wires": []
  },
  {
    "id": "local-mosquitto",
    "type": "mqtt-broker",
    "name": "Local Mosquitto",
    "broker": "localhost",
    "port": "1883",
    "clientId": "",
    "autoConnect": true,
    "usetls": false,
    "protocolVersion": "4",
    "keepalive": "60",
    "cleansession": true,
    "autoUnsubscribe": true,
    "birthTopic": "",
    "birthQos": "0",
    "birthPayload": "",
    "closeTopic": "",
    "closeQos": "0",
    "closePayload": ""
  }
]

```

Observed implementation evidence: the status update records this successful publisher output:

```

Connected to MQTT broker
Published message to: taxis/vehicle/VT001/telemetry

```

The same status update records No url specified in the Node-RED Debug panel. It is retained here as a limitation, not presented as a successful downstream test. No screenshot files or cloud deployment evidence were present in the project folder when this report was prepared.